Reverse Proxy & Rate Limiting (optional add-on)

Not in the 90-day table. Do the reverse proxy after Day 67 (Docker Compose) and the rate limiter after Day 57 (Redis cache). Each is about one day of work.

Doc 07 already lists "no rate limiting on `/auth/login`" as a known gap. These notes close it in two layers:

- **Nginx** in front of everything: one public entry point, coarse per-IP limits.
- **Spring filter** inside the app: per-user, per-endpoint limits that know who is logged in.

```
Client ──► Nginx :80/:443 ──► banking-app :8080          (/auth/**, /api/v1/**)
              │   (limit_req)    └─ RateLimitFilter ─► JwtAuthenticationFilter ─► controllers
              └──────────────► notification-service :8081  (/notifications/**)
                                    Redis :6379 ◄── shared rate-limit counters
```

Part A: Reverse proxy with Nginx

Why bother

- **Single entry point:** clients only see port 80/443. The app, notification-service, MySQL, Redis and Kafka are no longer published to the host.
- **TLS termination:** Nginx handles HTTPS, and Spring keeps serving plain HTTP inside the Docker network.
- **Path routing:** one host, two services (`/api` to banking-app, `/notifications` to notification-service).
- **Cheap protection:** body-size limits, timeouts and first-line rate limits run before a request ever reaches Java.
- **Scaling later:** an `upstream` block can list two app instances and Nginx load-balances them.

1. `nginx/nginx.conf` (new file at repo root)

```
events {}

http {
    # --- rate-limit zones (used in Part B, layer 1) ---
    limit_req_zone $binary_remote_addr zone=login_zone:10m rate=5r/m;
    limit_req_zone $binary_remote_addr zone=api_zone:10m   rate=20r/s;
    limit_req_status 429;

    upstream banking_app          { server banking-app:8080; }
    upstream notification_service { server notification-service:8081; }

    server {
        listen 80;
        client_max_body_size 1m;          # banking JSON bodies are tiny
        proxy_connect_timeout 5s;
        proxy_read_timeout    30s;

        # Overwrite (don't append) so a client can't inject a fake IP
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;

        location = /auth/login {
            limit_req zone=login_zone burst=3 nodelay;
            proxy_pass http://banking_app;
        }
        location /auth/ {
            proxy_pass http://banking_app;
        }
        location /api/ {
            limit_req zone=api_zone burst=40 nodelay;
            proxy_pass http://banking_app;
        }
        location /notifications/ {        # adjust to notification-service's real prefix
            proxy_pass http://notification_service;
        }
        location /actuator/ {
            deny all;                     # keep metrics internal; Prometheus scrapes inside Docker
        }
    }
}
```

`proxy_pass http://banking_app;` has no trailing slash, so the path is forwarded unchanged (`/api/v1/accounts/1` stays `/api/v1/accounts/1`). With a trailing slash, Nginx would strip the matched prefix. Getting this wrong is the most common Nginx mistake.

2. docker-compose.yml changes

```
  nginx:
    image: nginx:1.27-alpine
    ports: ["80:80"]
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on: [banking-app, notification-service]

  banking-app:
    # ...existing config...
    expose: ["8080"]          # was ports: ["8080:8080"]; now reachable only via nginx

  notification-service:
    expose: ["8081"]          # same idea
```

Keep `ports` on banking-app while you are still debugging. Remove it once traffic through Nginx works.

3. Make Spring trust the proxy (important for this codebase)

`AuditServiceImpl.resolveClientIp()` currently does this:

```
String forwardedFor = request.getHeader("X-Forwarded-For");
if (forwardedFor != null && !forwardedFor.isBlank()) {
    return forwardedFor.split(",")[0].trim();     // first entry = whatever the client sent
}
```

Without a proxy, anyone can send `X-Forwarded-For: 1.2.3.4` and the audit log will record 1.2.3.4. The same spoofed value would also let an attacker dodge an IP-based rate limiter.

The fix is to let Tomcat handle forwarded headers, then read only `getRemoteAddr()`:

```
# application-prod.properties
server.forward-headers-strategy=native
# Tomcat only honours X-Forwarded-* when the immediate caller matches internal-proxies.
# The default regex already covers private ranges (10.x, 172.16-31.x, 192.168.x, 127.x),
# which includes the Docker network.
```

```
private String resolveClientIp() {
    if (!(RequestContextHolder.getRequestAttributes() instanceof ServletRequestAttributes
attributes)) {
        return null;
    }
    return attributes.getRequest().getRemoteAddr();   // already the real client IP via
RemoteIpValve
}
```

Interview point: `X-Forwarded-For` is only trustworthy when it is set by a proxy you control. Either have the edge proxy overwrite it, as above, or read it from the right-hand side, skipping known proxies (Tomcat's RemoteIpValve does this). Never trust the left-most value.

4. Optional: HTTPS

```
# self-signed cert for local testing
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout nginx/certs/server.key -out nginx/certs/server.crt -subj "/CN=localhost"

server { listen 80;  return 301 https://$host$request_uri; }
server {
    listen 443 ssl;
    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    # ...same locations as above...
}
```

Mount `./nginx/certs` and publish `"443:443"`. On AWS (doc 08), the usual approach is to terminate TLS at the load balancer with an ACM certificate and keep Nginx on HTTP behind it.

Test checklist

- [ ] `docker compose up -d`, then `curl -i http://localhost/auth/login ...` returns the same response as hitting :8080 directly.
- [ ] `curl http://localhost:8080/...` fails once `ports` is removed from banking-app.
- [ ] `curl -H "X-Forwarded-For: 6.6.6.6" ...` through Nginx: the audit log shows the real IP (Docker gateway, e.g. 172.x), not 6.6.6.6.
- [ ] `curl http://localhost/actuator/health` returns 403.
- [ ] Sending a 2 MB body returns 413 from Nginx; the app never sees it.

Alternative: Spring Cloud Gateway

Spring Cloud Gateway is a Java-based gateway with route predicates and a Redis rate limiter built in. It is worth naming in interviews, but it adds a third Spring Boot service to maintain, and you would need to check that its version supports Spring Boot 4.1. For two services, Nginx is simpler and is what most teams put at the edge anyway.

Part B: Rate limiting

Algorithms (know these for interviews)

Algorithm	How it works	Trade-off
Fixed window	Counter per key per minute (`INCR`, expires at window end)	Simplest. Allows a burst of 2× the limit at window edges (end of one minute plus start of the next).
Sliding window log / counter	Counts requests in the last N seconds, not the calendar minute	Accurate, but uses more memory or Redis operations.
Token bucket	Bucket holds N tokens and refills at a fixed rate; each request takes one	Allows short bursts with a smooth average. Used by Bucket4j and the AWS API Gateway.
Leaky bucket	Requests queue and drain at a fixed rate	Produces smooth output. Nginx `limit_req` works like this; `burst` is the queue size.

Policies for this project

Endpoint	Key	Limit	Why
`POST /auth/login`	client IP	5 / minute	Slows password brute-force (no user is known yet)
`POST /auth/register`	client IP	3 / hour	Blocks fake-account spam
`POST /api/v1/transactions/transfer`, `/withdraw`	username	10 / minute	Limits damage from a stolen token or a buggy client loop
everything else under `/api/`	username	100 / minute	General fair use

A rejected request returns **429 Too Many Requests** with a `Retry-After: <seconds>` header and the usual `ErrorResponse` JSON body.

Layer 1: Nginx (already in the config above)

`limit_req_zone $binary_remote_addr zone=login_zone:10m rate=5r/m` tracks about 160k IPs in 10 MB. `burst=3 nodelay` lets 3 extra requests through immediately and rejects anything beyond that with 429. This layer is per-IP only and has no idea who the user is. Its job is to absorb floods cheaply.

Layer 2, option 1: Bucket4j in memory (start here)

```
<dependency>
    <groupId>com.bucket4j</groupId>
    <artifactId>bucket4j_jdk17-core</artifactId>
    <version>8.14.0</version>   <!-- check Maven Central for the latest 8.x -->
</dependency>
```

```
package org.example.enterprisedigitalbankingsystem.security;

public class RateLimitFilter extends OncePerRequestFilter {

    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {
        RateLimitPolicy policy = RateLimitPolicy.resolve(request);
        if (policy == null) {                       // unlimited path
            chain.doFilter(request, response);
            return;
        }
        String key = policy.name() + ":" + policy.keyFor(request);
        Bucket bucket = buckets.computeIfAbsent(key, k -> Bucket.builder()
                .addLimit(limit -> limit.capacity(policy.capacity())
                        .refillGreedy(policy.capacity(), policy.period()))
                .build());

        ConsumptionProbe probe = bucket.tryConsumeAndReturnRemaining(1);
        if (probe.isConsumed()) {
            response.setHeader("X-RateLimit-Remaining",
String.valueOf(probe.getRemainingTokens()));
            chain.doFilter(request, response);
        } else {
            long retryAfter = Math.max(1, probe.getNanosToWaitForRefill() / 1_000_000_000L);
            reject(request, response, retryAfter);
        }
    }

    private void reject(HttpServletRequest request, HttpServletResponse response, long retryAfter)
            throws IOException {
        response.setStatus(429);
        response.setHeader("Retry-After", String.valueOf(retryAfter));
        response.setContentType("application/json");
        response.getWriter().write(String.format(
                "{\"timestamp\":\"%s\",\"status\":429,\"error\":\"Too many requests, retry in %d
s\",\"path\":\"%s\"}",
                LocalDateTime.now(), retryAfter, request.getRequestURI()));
    }
}
```

```
public enum RateLimitPolicy {
    LOGIN(5, Duration.ofMinutes(1)),
    REGISTER(3, Duration.ofHours(1)),
    MONEY_MOVEMENT(10, Duration.ofMinutes(1)),
    GENERAL(100, Duration.ofMinutes(1));

    // constructor + capacity()/period() getters ...

    public static RateLimitPolicy resolve(HttpServletRequest req) {
        String path = req.getRequestURI();
        if (path.equals("/auth/login"))    return LOGIN;
        if (path.equals("/auth/register")) return REGISTER;
        if (path.equals("/api/v1/transactions/transfer")
                || path.equals("/api/v1/transactions/withdraw")) return MONEY_MOVEMENT;
        if (path.startsWith("/api/"))      return GENERAL;
        return null;
    }

    public String keyFor(HttpServletRequest req) {
        if (this == LOGIN || this == REGISTER) return req.getRemoteAddr();   // real IP after Part
A step 3
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        return (auth != null && auth.isAuthenticated() && !(auth instanceof
AnonymousAuthenticationToken))
                ? auth.getName() : req.getRemoteAddr();
    }
}
```

Limitation: the buckets live in one JVM's memory. If you run two app instances behind Nginx, each one allows the full limit, which doubles the effective rate. A restart also resets every counter. The map never shrinks either. That is acceptable for a demo; in production you would use a Caffeine cache with expiry or option 2 below.

Layer 2, option 2: Redis-backed (distributed, uses the Redis from Day 56)

This is a fixed window built on one atomic Lua script. It needs no extra library, because `spring-boot-starter-data-redis` is already there after Day 56.

```
@Component
public class RedisRateLimiter {

    private static final DefaultRedisScript<Long> INCR_WITH_TTL = new DefaultRedisScript<>("""
            local c = redis.call('INCR', KEYS[1])
            if c == 1 then redis.call('EXPIRE', KEYS[1], ARGV[1]) end
            return c
            """, Long.class);

    private final StringRedisTemplate redis;

    public RedisRateLimiter(StringRedisTemplate redis) { this.redis = redis; }

    /** @return seconds to wait, or 0 if allowed */
    public long check(RateLimitPolicy policy, String id) {
        long windowSec = policy.period().toSeconds();
        long window = Instant.now().getEpochSecond() / windowSec;
        String key = "rl:" + policy.name() + ":" + id + ":" + window;
        try {
            Long count = redis.execute(INCR_WITH_TTL, List.of(key), String.valueOf(windowSec));
            if (count != null && count > policy.capacity()) {
                return windowSec - (Instant.now().getEpochSecond() % windowSec);
            }
            return 0;
        } catch (DataAccessException e) {
            log.warn("Rate limiter unavailable, allowing request", e);   // fail-open
            return 0;
        }
    }
}
```

- **Why Lua:** a separate `INCR` then `EXPIRE` is two round trips. If the app crashes between them, the key never expires and that user is locked out forever. The script runs atomically on Redis.
- **Fail-open vs fail-closed:** if Redis is down, should requests be allowed? Fail-open (as above) keeps the bank usable, and Nginx layer 1 still gives some protection. Fail-closed is safer for `/auth/login` specifically. Pick one deliberately and be ready to justify it.
- Inspect live counters with `docker exec -it <redis> redis-cli KEYS "rl:*"` and `TTL <key>`.
- To use Bucket4j's token bucket across instances instead, it has a Redis/Lettuce module (`bucket4j_jdk17-lettuce`) that stores bucket state in Redis.

Wiring the filter into SecurityConfig

```
http
    // ...existing config...
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
    .addFilterAfter(rateLimitFilter, JwtAuthenticationFilter.class);   // runs after JWT, so
auth.getName() works
```

Order matters. Running after the JWT filter lets per-user policies see the username. Login and register are keyed by IP, so the order makes no difference for them.

Gotcha: if `RateLimitFilter` is a `@Component`, Spring Boot also registers it as a plain servlet filter, and that one runs *before* the security chain. `OncePerRequestFilter` then skips the security-chain copy, so the username is never available. Prevent this with:

```
@Bean
FilterRegistrationBean<RateLimitFilter> disableRateLimitAutoRegistration(RateLimitFilter f) {
    FilterRegistrationBean<RateLimitFilter> reg = new FilterRegistrationBean<>(f);
    reg.setEnabled(false);
    return reg;
}
```

Your existing `JwtAuthenticationFilter` is also a `@Component` and has the same double registration, so add a similar bean for it.

Related but different: account lockout

A rate limit slows down one IP. An attacker using many IPs against one account is caught by **lockout** instead: after 5 failed logins, set `User.lockedUntil = now + 15 min` and have `CustomUserDetails.isAccountNonLocked()` return false until then. Real banks use both. This is a common interview follow-up.

Test checklist

[ ] `for i in $(seq 1 7); do curl -s -o /dev/null -w "%{http_code}\n" -X POST localhost:8080/auth/login -H "Content-Type: application/json" -d '{"email":"x@x.com","password":"wrong"}'; done` returns five 401s, then 429s.

[ ] The 429 response has `Retry-After`. Waiting that long and retrying succeeds.

[ ] User A hitting the transfer limit does not block user B (the key is the username).

[ ] Redis option: with two app instances (`docker compose up --scale banking-app=2`), the limit is still enforced in total across both.

[ ] With Redis stopped, requests still go through (fail-open) and a WARN log appears.

[ ] MockMvc test: loop `perform(post("/auth/login"))` 6 times and assert the last one is `status().isTooManyRequests()`.

Interview questions

- **Where should rate limiting live?** In layers. The edge (Nginx, API gateway or WAF) is cheap and blocks floods by IP. The application knows the user and the business action, so it can enforce smarter limits.
- **Why Redis for the counters?** Shared state across instances, atomic `INCR`, built-in TTL, and it is fast enough to sit on every request.
- **Token bucket vs fixed window?** Token bucket allows controlled bursts and avoids the 2× spike at window edges. Fixed window is simpler and good enough for login throttling.
- **Why 429 and not 403?** 429 means "come back later" and includes `Retry-After`. 403 means you are never allowed. Clients and SDKs retry 429 with backoff.
- **Reverse proxy vs forward proxy vs load balancer?** A reverse proxy sits in front of servers and acts for them. A forward proxy sits in front of clients. A load balancer is a reverse proxy that spreads requests across several instances.
- **How do you get the real client IP behind a proxy?** Use `X-Forwarded-For`, trusting only hops you control. See Part A step 3.